数值线性代数　第 15 讲：稳定性漫谈

Lloyd N. Trefethen、David Bau III　中文阅读版

我们继续讨论稳定性，考察稳定与不稳定算法的例子. 随后将讨论一个把良态性和稳定性联系起来的基本思想：后向误差分析. 自 20 世纪 50 年代以来，它的力量已经在无数应用中得到证明.

浮点运算的稳定性

最简单的四个计算问题是 $+$、$-$、$\times$ 和 $\div$. 在算法选择方面没有太多可说的! 当然，我们通常会使用计算机提供的浮点运算 $\oplus$、$\ominus$、$\otimes$ 和 $\oslash$. 事实证明，公理 (13.5) 和 (13.7) 蕴含：这四个典型算法都是后向稳定的.

下面以减法为例说明，因为这是人们最容易担心不稳定的基本运算. 与例 12.3 一样，数据空间 X 是二维向量空间 $\mathbb{C}^2$，解空间 Y 是标量空间 $\mathbb{C}$. 由定理 14.1，我们无需指定这些空间上的范数.

对数据 $x = (x_1, x_2)^* \in X$，减法问题对应于函数

$$f(x_1, x_2) = x_1 - x_2,$$

所考虑的算法可以写成

$$\widetilde{f}(x_1, x_2) = \mathrm{fl}(x_1) \ominus \mathrm{fl}(x_2).$$

这个等式表示：先把 x_1 和 x_2 舍入为浮点数，再应用浮点减法. 由式 (13.5)，有

$$\mathrm{fl}(x_1) = x_1(1+\epsilon_1), \qquad \mathrm{fl}(x_2) = x_2(1+\epsilon_2), \qquad |\epsilon_1|, |\epsilon_2| \le \varepsilon_{\text{machine}}.$$

由式 (13.7)，有

$$\mathrm{fl}(x_1) \ominus \mathrm{fl}(x_2) = \big(\mathrm{fl}(x_1) - \mathrm{fl}(x_2)\big)(1+\epsilon_3), \qquad |\epsilon_3| \le \varepsilon_{\text{machine}}.$$

合并这些等式得到

$$\begin{aligned} \mathrm{fl}(x_1) \ominus \mathrm{fl}(x_2) &= [x_1(1+\epsilon_1) - x_2(1+\epsilon_2)](1+\epsilon_3) \\ &= x_1(1+\epsilon_1)(1+\epsilon_3) - x_2(1+\epsilon_2)(1+\epsilon_3) \\ &= x_1(1+\epsilon_4) - x_2(1+\epsilon_5), \end{aligned}$$

其中

$$|\epsilon_4|, |\epsilon_5| \le 2\varepsilon_{\text{machine}} + O(\varepsilon_{\text{machine}}^2)$$

（见习题 14.2）. 换句话说，计算结果

$$\widetilde{f}(x) = \text{fl}(x_1) \ominus \text{fl}(x_2)$$

恰好等于 $\widetilde{x}_1 - \widetilde{x}_2$，其中 $\widetilde{x}_1, \widetilde{x}_2$ 满足

$$\frac{|\widetilde{x}_1 - x_1|}{|x_1|} = O(\varepsilon_{\text{machine}}), \qquad \frac{|\widetilde{x}_2 - x_2|}{|x_2|} = O(\varepsilon_{\text{machine}}).$$

隐含常数取任意 $C > 2$ 都足够. 对 $\mathbb{C}^2$ 上的任意范数，这就推出式 (14.5).

更多例子

例 15.1：内积

给定向量 $x, y \in \mathbb{C}^m$，希望计算内积

$$\alpha = x^* y.$$

显然的算法是：使用 $\otimes$ 计算逐项乘积 $\widetilde{x}_i \otimes \widetilde{y}_i$，再使用 $\oplus$ 将它们相加，得到计算结果 $\widetilde{\alpha}$. 可以证明，这个算法是后向稳定的；第 17 讲会隐含地完成这一证明. □

例 15.2：外积

另一方面，设希望对向量 $x \in \mathbb{C}^m$、$y \in \mathbb{C}^n$ 计算秩一外积

$$A = xy^*.$$

显然的算法是：使用 $\otimes$ 计算 mn 个乘积 $\widetilde{x}_i \otimes \widetilde{y}_j$，并将它们收集为矩阵 $\widetilde{A}$. 这个算法是稳定的，但不是后向稳定的. 原因在于，矩阵 $\widetilde{A}$ 极不可能恰好具有秩 1，因此一般不能写成

$$\widetilde{A} = (x + \delta x)(y + \delta y)^*.$$

一般而言，对于解空间 Y 的维数大于问题空间 X 维数的问题，后向稳定性是罕见的. □

例 15.3

假设使用 $\oplus$ 计算 $x + 1$，其中 $x \in \mathbb{C}$：

$$\widetilde{f}(x) = \text{fl}(x) \oplus 1.$$

这个算法稳定，但不是后向稳定的. 原因是，当 $x \approx 0$ 时，加法 $\oplus$ 会引入大小为 $O(\varepsilon_{\text{machine}})$ 的绝对误差. 相对于 x 的大小，这些误差是无界的，因此不能解释为数据发生了小的相对扰动. 这个例子说明，后向稳定性是一种相当特殊的性质：在某些情形下它是合理目标，在另一些情形下则不是. 注意，如果问题是对数据 x, y 计算 $x + y$，那么对应算法就是后向稳定的. □

例 15.4

对于计算 $\sin x$ 或 $\cos x$ 的计算机程序或计算器，合理的期待是什么？答案仍然是稳定性，而不是后向稳定性. 对于 $\cos x$，这来自前一个例子中的事实 $\cos 0 \neq 0$. 对于 $\sin x$ 和 $\cos x$ 二者，函数在某些点导数为零这一事实也排除了后向稳定性.

例如，设计算机计算

$$f(x) = \sin x, \qquad x = \frac{\pi}{2} - \delta, \qquad 0 < \delta \ll 1.$$

假设我们足够幸运，得到的计算结果正好是正确答案在浮点系统中的舍入值：

$$\widetilde{f}(x) = \mathrm{fl}(\sin x).$$

由于

$$f'(x) = \cos x \approx \delta,$$

所以存在某个 $\widetilde{x}$ 使 $\widetilde{f}(x) = f(\widetilde{x})$，并且

$$\widetilde{x} - x \approx \frac{\widetilde{f}(x) - f(x)}{\delta} = O\left(\frac{\varepsilon_{\text{machine}}}{\delta}\right).$$

由于 δ 可以任意小，这个后向误差不可能具有 $O(\varepsilon_{\text{machine}})$ 的量级. □

一个不稳定算法

以上都是玩具例子. 下面给出一个更实质的例子：使用特征多项式求矩阵的特征值.

由于 z 是 A 的特征值当且仅当 $p(z) = 0$，其中

$$p(z) = \det(zI - A)$$

是特征多项式，所以 p 的根就是 A 的特征值（第 24 讲）. 这启发出如下的特征值计算方法：

1. 求出特征多项式的系数；
2. 求这个多项式的根.

这个算法不仅后向不稳定，而且是不稳定的，因此不应使用. 即使求特征值本身是良态问题，它也可能产生远大于 $\varepsilon_{\text{machine}}$ 的相对误差.

不稳定性出现在第二步的求根过程中. 正如例 12.5 所见，给定多项式系数后求根通常是病态问题. 因此，特征多项式系数中的小误差在求根时往往会被放大，即使求根过程本身可以做到完全精确.

例如，设 $A = I$ 是 2×2 单位矩阵. A 的特征值对矩阵元素的扰动不敏感，稳定算法应当能够以 $O(\varepsilon_{\text{machine}})$ 的误差计算它们. 然而，上述算法产生的误差量级是 $\sqrt{\varepsilon_{\text{machine}}}$. 原因是特征多项式

$$p(x) = x^2 - 2x + 1,$$

正如例 12.5 所示，其系数计算预计会产生 $O(\varepsilon_{\text{machine}})$ 量级的误差，而这些误差会使根发生 $\sqrt{\varepsilon_{\text{machine}}}$ 量级的变化. 例如，如果 $\varepsilon_{\text{machine}} = 10^{-16}$，计算得到的特征多项式的根可能偏离真实特征值约 10^{-8}，也就是损失 8 位准确数字.

在亲自尝试这个计算之前，还需要更诚实地说明一点. 如果直接用上述算法计算 2×2 单位矩阵的特征值，你很可能发现完全没有误差，因为 $x^2 - 2x + 1$ 的系数和根都是计算机能够精确表示的小整数. 但是，如果对一个略有扰动的矩阵进行实验，例如

$$A = \begin{bmatrix} 1 + 10^{-14} & 0 \\ 0 & 1 \end{bmatrix},$$

计算得到的特征值就会与真实特征值相差预期的 $\sqrt{\varepsilon_{\text{machine}}}$ 量级. 试试看!

后向稳定算法的准确性

设我们有一个用于问题 $f : X \to Y$ 的后向稳定算法 $\widetilde{f}$. 它给出的结果会准确吗? 答案取决于问题的条件数 $\kappa = \kappa(x)$. 如果 $\kappa(x)$ 较小，结果在相对意义下会很准确；但如果 $\kappa(x)$ 很大，准确性就会按比例下降.

定理 15.1. 设后向稳定算法用于求解问题 $f : X \to Y$，问题在数据 x 处的条件数为 $\kappa = \kappa(x)$. 假设计算机满足公理 (13.5) 和 (13.7)，则相对误差满足

$$\frac{\|\widetilde{f}(x) - f(x)\|}{\|f(x)\|} = O\big(\kappa(x)\varepsilon_{\text{machine}}\big). \tag{15.1}$$

证明. 由后向稳定性的定义 (14.5)，存在某个 $\widetilde{x} \in X$，使得

$$\widetilde{f}(x) = f(\widetilde{x}), \qquad \frac{\|\widetilde{x} - x\|}{\|x\|} = O(\varepsilon_{\text{machine}}).$$

由 $\kappa(x)$ 的定义 (12.5)，这意味着

$$\frac{\|\widetilde{f}(x) - f(x)\|}{\|f(x)\|} \le \big(\kappa(x) + o(1)\big)\frac{\|\widetilde{x} - x\|}{\|x\|}, \tag{15.2}$$

其中 $o(1)$ 表示当 $\varepsilon_{\text{machine}} \to 0$ 时趋于零的量. 合并这两个界就得到式 (15.1). □

后向误差分析

刚才证明定理 15.1 的过程称为后向误差分析. 我们通过两个步骤得到准确度估计. 一步是研究问题的条件性，另一步是研究算法的稳定性. 结论是：如果算法稳定，那么最终准确度就会反映问题的条件数.

从数学上说，这个过程很直接，但如果让一个没有准备的人分析数值算法，这当然不会是他首先想到的办法. 第一个想法通常是前向误差分析：估计计算每一步引入的舍入误差，并设法累计这些误差在各步之间如何复合.

经验表明，对于数值线性代数中的大多数算法，前向误差分析比后向误差分析更难. 事后看来，原因并不难解释. 假设在计算机上使用一个久经检验的算法，例如求解 $Ax = b$. 已知当 A 病态时，得到的结果会始终不那么准确（见第 22 讲）. 那么，前向误差分析如何捕捉这一现象呢？A 的条件数是一个非常全局的性质，在求解 $Ax = b$ 时所涉及的单次浮点运算层面上几乎不可见.（下一讲会通过一个例子把这一点表现得更明显.）然而，前向分析无论如何都必须检测到这个条件数，才能最终得到正确结果.

简言之，对于大多数问题，最好的算法一般也不会做得比“对略有扰动的数据计算精确解”更好. 后向误差分析正是一种与这种后向现实相适应的推理方法.

习题

15.1 下面每个问题都描述了一个在满足公理 (13.5) 和 (13.7) 的计算机上实现的算法. 对每一个问题，判断算法是后向稳定、稳定但非后向稳定，还是不稳定，并证明结论，或者至少给出有说服力的理由. 务必遵循正文中的定义.

(a) 数据：$x \in \mathbb{C}$. 解：$2x$，计算为 $x \oplus x$.

(b) 数据：$x \in \mathbb{C}$. 解：x^2，计算为 $x \otimes x$.

(c) 数据：$x \in \mathbb{C} \setminus \{0\}$. 解：$1$，计算为 $x \oslash x$.（满足式 (13.6) 的机器会给出完全正确的答案，但我们的定义基于较弱的条件 (13.7).）

(d) 数据：$x \in \mathbb{C}$. 解：0，计算为 $x \ominus x$.（同样，真实机器可能比基于式 (13.7) 的定义表现得更好.）

(e) 无数据. 解：e，从左到右使用 $\otimes$ 和 $\oplus$ 对

$$\sum_{k=0}^{\infty} \frac{1}{k!}$$

求和，直到某一项的大小小于 $\varepsilon_{\text{machine}}$ 时停止.

(f) 无数据. 解：e，使用上述同一算法，但从右到左对级数求和.

(g) 无数据. 解：π，穷举搜索区间 $[3, 4]$ 中最小的浮点数 x，使得

$$s(x) \otimes s(x') < 0.$$

这里 $s(x)$ 是在给定区间内稳定计算 $\sin(x)$ 的算法，x' 表示浮点系统中紧随 x 之后的下一个浮点数.

15.2 考虑计算矩阵完整 SVD 的算法. 该问题的数据是矩阵 A，解是三个矩阵 U（酉矩阵）、Σ（对角矩阵）和 V（酉矩阵），满足

$$A = U\Sigma V^*.$$

这里说的是显式矩阵 U 和 V，而不是由反射器乘积给出的隐式表示.

(a) 解释这个算法后向稳定意味着什么.

(b) 事实上，由一个简单原因，这个算法不可能后向稳定. 解释原因.

(c) 幸运的是，计算 SVD 的标准算法（第 31 讲）是稳定的. 解释对于这类算法，稳定性意味着什么.